

Rapport du modèle client serveur du jeux "Plus ou Moin"

BOUVY Paoline

16 septembre 2026

Table des matières

1	Introduction	3
2	Objectif	3
3	Technologie utilisées	3
4	Partie serveur	3
4.1	Connection avec les clients	3
4.2	Codage du jeux :	3
4.2.1	Vérification de la réponse client	3
4.2.2	Savoir si le client à gagner	4
5	Partie client	4
5.1	Connection avec le serveur	4
5.2	Codage de la partie jeux	4
6	Axe d'Amélioration	4

1 Introduction

Votre texte commence ici.

2 Objectif

Ce projet a pour objectif de montrer le fonctionnement du modèle client-serveur dans un but pédagogique. L'idée est de mettre en place une activité où l'ordinateur de l'enseignant joue le rôle du serveur et où les ordinateurs des élèves jouent le rôle des clients. Par conséquent, les élèves peuvent observer qu'ils ne disposent pas du code du jeu, tout en ayant la possibilité d'y jouer. Ils peuvent également remarquer concrètement les différentes requêtes envoyées au serveur ainsi que les réponses de celui-ci.

3 Technologie utilisées

Pour ce projet, j'ai utilisé Python, car c'est un langage utilisé aujourd'hui dans l'enseignement au lycée. Pour mettre en place une communication TCP sur un même réseau, j'ai utilisé la bibliothèque socket de Python. Ensuite, pour créer le jeu, j'ai utilisé la bibliothèque random afin d'importer la fonction randint, qui permet de générer un nombre aléatoire pour chaque partie.

4 Partie serveur

4.1 Connection avec les clients

Tout d'abord, pour établir la connexion entre le client et le serveur, j'ai initialisé un socket, puis je l'ai associé à l'adresse IP et au port choisis. Ensuite, j'attends qu'un joueur se connecte grâce à la méthode socket.accept(). Le serveur attend ensuite les réponses du client avec la méthode socket.recv().

Cependant, comme ces méthodes sont des appels bloquants et que je souhaitais pouvoir arrêter le serveur proprement, notamment à l'aide d'un Ctrl+C dans le terminal, j'ai mis en place un délai d'attente (timeout) défini par une constante.

J'ai ensuite utilisé des blocs try...except afin de gérer les différentes erreurs, comme l'expiration du délai d'attente, la déconnexion du client ou encore l'arrêt volontaire du serveur. Grâce à cette méthode, le code ne reste pas bloqué et le serveur demeure opérationnel pour plusieurs clients.

4.2 Codage du jeux :

4.2.1 Vérification de la réponse client

Pour la partie jeu, j'ai créé une fonction. Tout d'abord, j'ai initialisé le nombre secret à l'aide de la fonction randint(), puis une variable qui compte le nombre d'essais et un booléen permettant de savoir s'il faut continuer ou non. La boucle du jeu dépend donc de la valeur de cette variable booléenne. Comme précédemment, j'ai mis en place un bloc try...except pour gérer le délai d'attente. Cela permet d'arrêter le serveur proprement si le client met trop de temps à répondre, s'il ferme la connexion ou si une autre erreur

survient.

Premièrement, le serveur attend la proposition du client. La variable `essais` est ensuite incrémentée, puis la proposition est testée afin de vérifier si elle respecte les critères requis. Je vérifie d'abord qu'il ne s'agit pas d'une chaîne de caractères grâce à la méthode `variable.isdigit()`. Ensuite, je convertis la variable `proposition` en entier, notamment dans le cas où l'utilisateur aurait saisi un nombre réel. Enfin, je vérifie si la proposition se trouve bien entre les bornes du jeu.

Si la réponse de l'utilisateur ne respecte pas les critères, un essai est tout de même comptabilisé et l'utilisateur peut saisir une nouvelle proposition.

4.2.2 Savoir si le client à gagner

Pour vérifier si la proposition du client correspond au nombre mystère, celle-ci est testée à l'aide de blocs `if`, `elif` et `else`. Selon chaque cas, le client reçoit une indication, comme « plus grand » ou « plus petit », puis il peut proposer un nouveau nombre. La variable `essais` est alors incrémentée de 1.

En revanche, s'il a gagné, la valeur du booléen passe à `False`, ce qui permet de sortir de la boucle du jeu.

5 Partie client

5.1 Connection avec le serveur

Tout d'abord, le client établit une connexion avec le serveur, puis il attend la réception d'un booléen afin de lancer la boucle de réponse nécessaire au déroulement du jeu. Le client indique ensuite s'il souhaite jouer ou non. S'il accepte, la fonction du jeu est exécutée ; sinon, la connexion est fermée.

Comme pour la partie serveur, un délai d'attente (`timeout`) est mis en place afin d'éviter les appels bloquants et de pouvoir arrêter proprement le programme à l'aide de la combinaison `Ctrl+C`.

5.2 Codage de la partie jeux

Dans la partie jeu côté client, le client attend tout d'abord le message du serveur, puis il peut saisir sa réponse.

Cette réponse est soumise à plusieurs tests. Tout d'abord, on vérifie qu'il s'agit bien d'un nombre grâce à la méthode `.isdigit()`. Ensuite, on vérifie qu'elle se trouve bien entre les constantes `BORNE INF` et `BORNE MAX`, qui définissent l'intervalle dans lequel se situe le nombre secret.

Le serveur indique ensuite au client si le nombre secret est plus grand ou plus petit que sa proposition. Cette procédure se répète dans une boucle jusqu'à ce que le client trouve la bonne réponse.

6 Axe d'Amélioration

Comme les parties consacrées à la connexion et au codage du jeu sont séparées en plusieurs fonctions, il serait possible de mettre en place un menu permettant de choisir entre différents types de jeux.